

Lecture 2: Notations and Core Concepts

S. Voloshynovskiy

This lecture consists of the following parts:

- ① Part 1: Basic Notations and image representation**
- ② Part 2: Foundations of Deep Learning (DL): Self-Supervised Learning for Feature Extraction**
- ③ Part 3: Notations for digital watermarking and perceptual hashing**
- ④ Part 4: Foundations of Signal Detection in Noise**

Why is this foundational part crucial for our learning process?

- **Unified Understanding:**

- Provides a **common mathematical background** for all topics in this course.
- Ensures consistency in how we approach problems across different domains.

- **Four Thematic Blocks:**

- **Digital Watermarking:** Embedding and detecting robust marks in content.
- **Perceptual Hashing:** Generating compact and robust representations.
- **Adversarial Robustness:** Analyzing and defending against adversarial attacks.
- **Privacy Protection Techniques:** Ensuring data privacy and regulatory compliance.

Part 1: Basic notations and image representation

Part 1.1: Notations for image representation

Basic Notations for Images

- **x**: Original image.
- **y**: Stego image containing the watermark.
- **w**: Watermark.
- **k**: Secret key used for embedding or decoding.
- **m**: Multi-bit message ($\mathbf{m} \in \{0,1\}^L$, where L is the message length).
- Images will have dimensions:

$$H \times W \times C$$

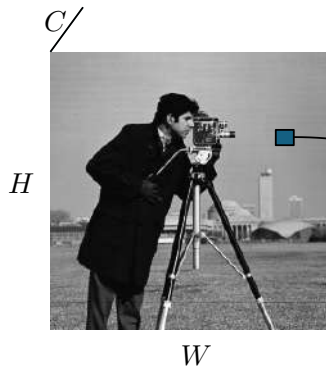
where:

- H : Height of the image (in pixels).
- W : Width of the image (in pixels).
- C : Number of color channels ($C = 1$ for grayscale, $C = 3$ for RGB).
- Each pixel $x_i \in \mathcal{X}$, where:

$$\mathcal{X} = \{0, 1, \dots, 255\}$$

(assuming 8-bit depth per channel).

Image notations



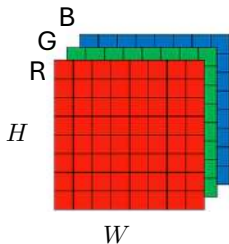
$$H \times W$$

$$x_i \in \mathcal{X} = \{0, \dots, 255\}$$

$$8\text{bits} : 2^8 = 256$$

$$\text{Total size: } (H \times W) \times 256$$

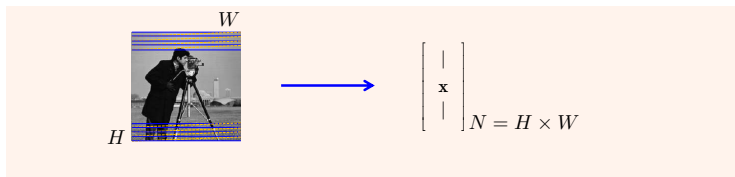
Image notations



$$x_i^{R,G,B} \in \mathcal{X} = \{0, \dots, 255\}$$

Total size: $(H \times W) \times 3 \times 256$

Vectorized Notations and Python Alignment



- Images can be flattened into vectors for mathematical operations:

$$\mathbf{x} \in \mathbb{R}^{H \cdot W}$$

where:

- $H \cdot W = M$: Total number of pixels for grayscale images.
- For RGB images:

$$\mathbf{x}_{\text{RGB}} \in \mathbb{R}^{3 \cdot H \cdot W}$$

- **Remark:** In Python, bold notation cannot be directly used. The context will clarify the dimensions being referred to.

Sources of Images

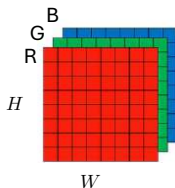
- Modern images are typically captured by mobile devices:
 - Common resolutions:
 - 12-24 MP (e.g., smartphones).
 - 108 MP (e.g., some models of Samsung cameras).
- Images undergo preprocessing:
 - From Bayer pattern to RGB via demosaicking.
 - RGB images are standard for multimedia processing.
- Input format for this class:

RGB images of size $H \times W \times 3$

Images from modern phones



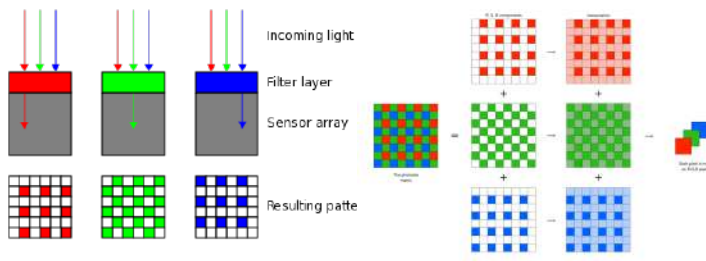
108 Mpx



12 Mpx

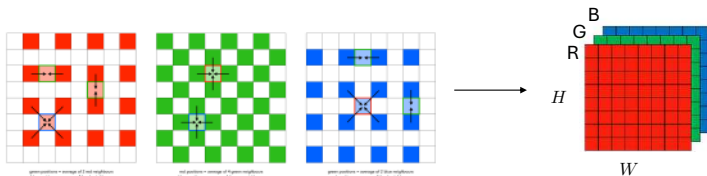
Demosaicing

Demosaicing



[https://pixelcraft.photo.blog/2023/09/06/
from-photosites-to-pixels-iv-the-demosaicing-process/](https://pixelcraft.photo.blog/2023/09/06/from-photosites-to-pixels-iv-the-demosaicing-process/)

Demosaicing



Part 1.2: Notations for perceptual quality metrics and image/vector similarity

Perceptual Quality Metrics

- **Mean Squared Error (MSE)** between $\mathbf{x}$ and $\mathbf{y}$:

$$\text{MSE}(\mathbf{x}, \mathbf{y}) = \frac{1}{M} \|\mathbf{x} - \mathbf{y}\|_2^2$$

- **Peak Signal-to-Noise Ratio (PSNR)**:

$$\text{PSNR} = 10 \cdot \log_{10} \left(\frac{\text{MAX}^2}{\text{MSE}(\mathbf{x}, \mathbf{y})} \right)$$

- **Weighted MSE (WMSE)**:

$$\text{WMSE}(\mathbf{x}, \mathbf{y}) = \frac{1}{M} \sum_{i=1}^M w_i \cdot (x_i - y_i)^2$$

where w_i is a weighting function (to be introduced later).

- **Weighted PSNR (WPSNR)**:

$$\text{WPSNR} = 10 \cdot \log_{10} \left(\frac{\text{MAX}^2}{\text{WMSE}(\mathbf{x}, \mathbf{y})} \right)$$

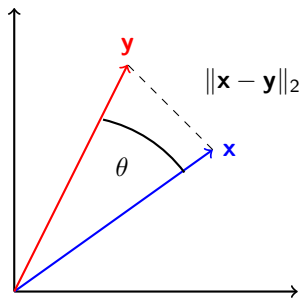
Metrics of Vector Proximity

- **Inner Product:**

$$\mathbf{x}^T \mathbf{y} = \sum_{i=1}^M x_i y_i$$

- **Cosine Similarity:**

$$\cos(\theta) = \frac{\mathbf{x}^T \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|}$$



Part 1.3: Notations for image transform domain representation

Overview of Image Transformations

This lecture will cover the following transformations:

① Bit-Plane Image Decomposition:

- Binary representation of image pixels.

② Orthogonal Family of Transforms:

- Examples include discrete Fourier transform (DFT) and discrete cosine transform (DCT).

③ Learnable Transforms:

- Principal Component Analysis (PCA).

④ ML/DL Learnable Transforms:

- Feature extraction that is invariant to image processing and geometric transformations.

Bit-Plane Decomposition

Bit-plane representation:

- Decomposes an image into binary planes corresponding to individual bits of its pixel values.
- For an 8-bit grayscale image, each pixel x is represented as:

$$x = x_0 \cdot 2^0 + x_1 \cdot 2^1 + x_2 \cdot 2^2 + \dots + x_7 \cdot 2^7,$$

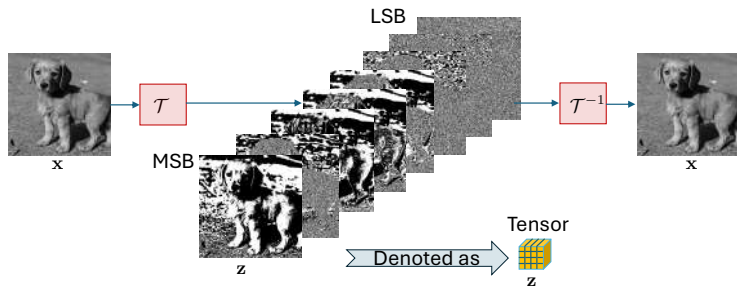
where $x_i \in \{0, 1\}$ is the i -th bit of the pixel value.

- This produces 8 binary images, one for each bit-plane.

Use Case:

- Highlights the contribution of each bit to the overall pixel value.
- Useful in compression, watermarking, and image analysis.

Bit-Plane Decomposition



Orthogonal Transformations: Overview

General Definition

$$\mathbf{z} = \mathcal{T}(\mathbf{x}), \quad \mathbf{x} = \mathcal{T}^{-1}(\mathbf{z}),$$

where $\mathcal{T}$ is the transform operator, and $\mathcal{T}^{-1}$ is its inverse.

Properties of Orthogonal Transformations:

- **Orthonormality:** Transform matrix $\mathcal{T}$ satisfies:

$$\mathcal{T}^\top \mathcal{T} = \mathcal{T} \mathcal{T}^\top = \mathbf{I}.$$

- **Lossless Transformation:** Orthogonal transforms allow perfect reconstruction of data between the original and transform domains.

Examples of Orthogonal Transforms:

- Discrete Fourier Transform (DFT)
- Discrete Cosine Transform (DCT)
- Discrete Wavelet Transform (DWT)

Orthogonal Transformations: Overview

General Definition:

$$\mathbf{z} = \mathcal{T}(\mathbf{x}), \quad \mathbf{x} = \mathcal{T}^{-1}(\mathbf{z}),$$

where $\mathcal{T}$ is the transform operator, and $\mathcal{T}^{-1}$ is its inverse.

Key Insight

Orthogonal transforms decompose signals into frequency components. The transformed domain enables separating low and high-frequency content, crucial for feature extraction and signal analysis.

Discrete Fourier Transform (DFT)

Definition

$$z[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-j2\pi kn/N}, \quad k = 0, \dots, N-1.$$

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} z[k] \cdot e^{j2\pi kn/N}.$$

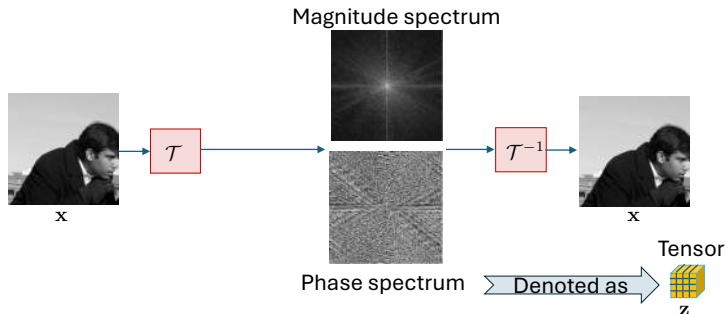
Core Properties of DFT:

- **Linearity:** The transform is linear.
- **Energy Preservation:** Total energy in the signal is preserved in the transform domain.
- **Periodicity:** The DFT is periodic with period N .
- **Magnitude Invariance:** The magnitude of the DFT is invariant under translation.

Applications of DFT:

- Signal analysis in the frequency domain.
- Filtering and denoising.

Discrete Fourier Transform (DFT)



Discrete Cosine Transform (DCT)

Definition

$$z[k] = \alpha_k \sum_{n=0}^{N-1} x[n] \cdot \cos \left(\frac{\pi}{N} (n + 0.5) k \right),$$

$$x[n] = \sum_{k=0}^{N-1} \alpha_k z[k] \cdot \cos \left(\frac{\pi}{N} (n + 0.5) k \right),$$

where:

$$\alpha_k = \begin{cases} \sqrt{\frac{1}{N}}, & k = 0, \\ \sqrt{\frac{2}{N}}, & \text{otherwise.} \end{cases}$$

Applications of DCT:

- JPEG and MPEG compression for images and videos.
- Signal and multimedia forensics.
- Watermarking and content-based retrieval.

Three Approaches to Applying DCT

1. Full-Frame DCT:

- The DCT is applied to the entire image as a single unit.
- Produces a frequency representation of the whole image.
- Computationally intensive for large images but provides global frequency information.

$$\mathbf{z} = \text{DCT}(\mathbf{x}), \quad \mathbf{x} \in \mathbb{R}^{H \times W}.$$

2. Block-Wise DCT (e.g., 8x8):

- The image is divided into non-overlapping blocks of size 8×8 .
- DCT is applied independently to each block.
- Used in compression algorithms like JPEG, focusing on local frequency content.

$$\mathbf{z}_{i,j} = \text{DCT}(\mathbf{x}_{i,j}), \quad \mathbf{x}_{i,j} \in \mathbb{R}^{8 \times 8}.$$

3. Sampling-Based DCT:

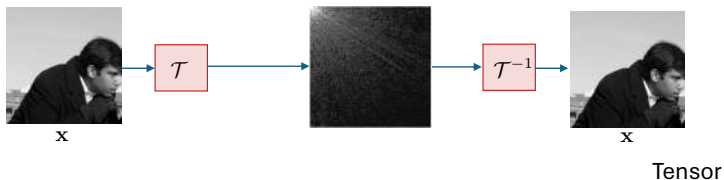
- The image is sampled every 8th pixel in both dimensions to create 64 downsampled images. Each downsampled image has dimensions $H/8 \times W/8$.
- DCT is applied to each of these downsampled images.
- Captures frequency content of the sampled images.

$$\mathbf{z}_{i,j} = \text{DCT}(\mathbf{x}^{(i,j)}), \quad \mathbf{x}^{(i,j)} \in \mathbb{R}^{H/8 \times W/8}.$$

Discrete Cosine Transform (DCT)

Full 2D frame DCT:

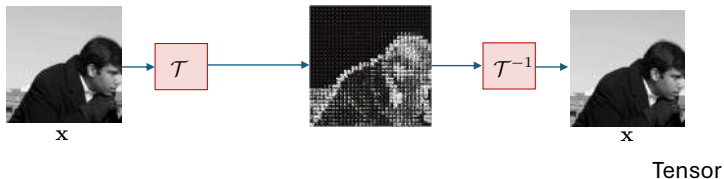
$$\mathbf{z} = \text{DCT}(\mathbf{x}), \quad \mathbf{x} \in \mathbb{R}^{H \times W}.$$



Discrete Cosine Transform (DCT)

8x8 block-wise 2D DCT:

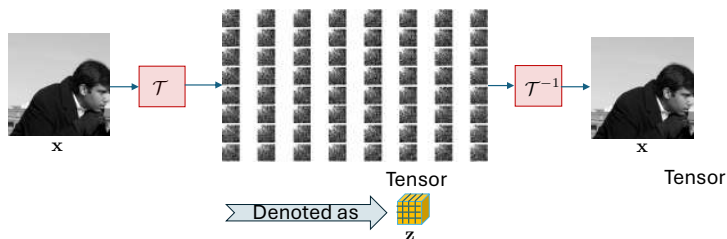
$$\mathbf{z}_{i,j} = \text{DCT}(\mathbf{x}_{i,j}), \quad \mathbf{x}_{i,j} \in \mathbb{R}^{8 \times 8}.$$



Discrete Cosine Transform (DCT)

Tensor 2D DCT:

$$\mathbf{z}_{i,j} = \text{DCT}(\mathbf{x}^{(i,j)}), \quad \mathbf{x}^{(i,j)} \in \mathbb{R}^{H/8 \times W/8}.$$



Discrete Wavelet Transform (DWT)

Definition:

- The Discrete Wavelet Transform decomposes a signal $\mathbf{x}$ into a set of wavelet coefficients $\mathbf{z}$, capturing both frequency and time (or spatial) localization.
- For a signal $\mathbf{x}$, the decomposition can be expressed as:

$$\mathbf{z} = \mathcal{T}(\mathbf{x}),$$

where $\mathcal{T}$ applies scaling (low-pass filtering) and wavelet (high-pass filtering) operations recursively.

Core Properties of DWT:

- **Multi-Resolution Analysis:** Captures details at various levels of resolution.
- **Energy Preservation:** Preserves the signal energy during transformation.
- **Compact Support:** Provides localized information in both time and frequency domains.
- **Reversibility:** Allows perfect reconstruction using the inverse DWT.

Applications of DWT:

- Image compression (e.g., JPEG 2000).
- Signal and audio denoising.

Discrete Wavelet Transform (DWT)



Learnable Transform: Principal Component Analysis (PCA)

General Definition

$$\mathbf{z} = \mathcal{T}(\mathbf{x}), \quad \mathbf{x} = \mathcal{T}^{-1}(\mathbf{z}),$$

where $\mathcal{T}$ is derived from data and is linear.

Steps:

① Collect N samples $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N$.

② Compute the mean:

$$\bar{\mathbf{x}} = \frac{1}{N} \sum_{i=1}^N \mathbf{x}_i.$$

③ Subtract the mean to center the data:

$$\mathbf{x}'_i = \mathbf{x}_i - \bar{\mathbf{x}}.$$

④ Compute the covariance matrix:

$$\mathbf{K}_{\mathbf{x}\mathbf{x}} = \frac{1}{N} \sum_{i=1}^N \mathbf{x}'_i \mathbf{x}'_i{}^{\top}.$$

Learnable Transform: Principal Component Analysis (PCA)

General Definition

$$\mathbf{z} = \mathcal{T}(\mathbf{x}), \quad \mathbf{x} = \mathcal{T}^{-1}(\mathbf{z}),$$

where $\mathcal{T}$ is derived from data and is linear.

Steps (cont.):

- 1 Perform eigenvalue decomposition:

$$\mathbf{K}_{xx} = \mathbf{U}\mathbf{\Sigma}\mathbf{U}^T,$$

where $\mathbf{U}$ contains eigenvectors and $\mathbf{\Sigma}$ is a diagonal matrix of eigenvalues.

- 2 Transform data into the principal component space:

$$\mathbf{z} = \mathbf{U}^T \mathbf{x}.$$

Key Properties of PCA Transformation

Matrix Representation:

- The PCA transformation is represented as:

$$\mathcal{T}(\mathbf{x}) = \mathbf{U}^\top \mathbf{x},$$

where $\mathbf{U}$ contains the eigenvectors of the covariance matrix $\mathbf{K}_{xx}$.

- The inverse transformation is:

$$\mathcal{T}^{-1}(\mathbf{z}) = \mathbf{U}\mathbf{z}.$$

Orthonormality of $\mathbf{U}$:

- The eigenvector matrix $\mathbf{U}$ satisfies:

$$\mathbf{U}^\top \mathbf{U} = \mathbf{U}\mathbf{U}^\top = \mathbf{I}.$$

- This property ensures that PCA is a fully invertible transformation.

Key Properties of PCA Transformation

Use of PCA:

- PCA is used for the data decorrelation and compaction.
- PCA might be used for a dimensionality reduction into a lower-dimensional subspace spanned by the eigenvectors of the covariance matrix $\mathbf{K}_{xx}$.
- PCA is most effective when the data can be well-approximated by a multidimensional Gaussian distribution.

Limitation of PCA

- If the data lies on a non-linear manifold, PCA may fail to capture its structure effectively.
- In such cases, more complex transformations, such as deep learning-based approaches, are required.

Overview:

- Modern deep learning (DL) techniques learn transformations invariant to:
 - Image processing (e.g., compression, noise).
 - Geometric transformations (e.g., rotation, scaling).
- These transformations aim to extract robust features for tasks such as classification, detection, and retrieval.

Key Examples:

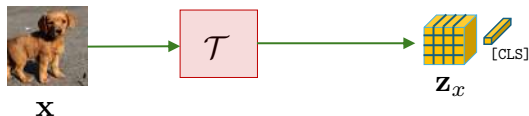
- Convolutional Neural Networks (CNNs) for spatial feature extraction.
- Self-Supervised Learning (SSL) for invariant representations.
- Learned embeddings used for robust similarity comparison.

Applications:

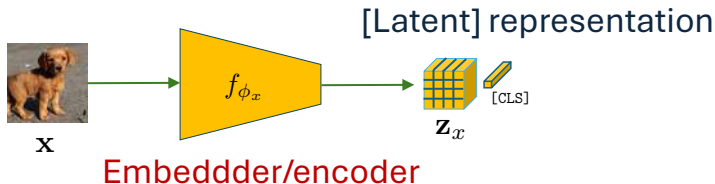
- Content-based retrieval.
- Image watermarking.
- Robust multimedia security.

**Part 2: Foundations of Deep Learning (DL):
self-supervised learning for feature extraction**

Foundation Models



Transform = Embedder/encoder



Transition to Machine-Learnable Transform Domains

From Handcrafted and Linear Transforms to Machine-Learnable Representations

- **Handcrafted and Linear Transforms:**

- Examples: PCA, Fourier, DCT.
- Characteristics: Orthogonal, invertible, and designed for specific properties (e.g., energy compaction).

- **Machine-Learnable Transform Domains:**

- Built using learnable models (e.g., deep neural networks, foundation models).
- Designed to produce a **latent space** that is:
 - **Representative:** Captures meaningful features of the data.
 - **Robust:** Resistant to noise and variations.
 - **Invariant:** Stable under image processing and geometrical transformations.
- Serves as a foundation for:
 - Watermarking and robust embedding.
 - Perceptual hashing.
 - Compression and other downstream tasks.

Transition to Machine-Learnable Transform Domains

From Handcrafted and Linear Transforms to Machine-Learnable Representations

- **Key Differences from Linear Learnable Transforms (PCA):**
 - **No Inverse Transform:** Unlike orthogonal transforms like PCA, the latent space cannot be perfectly reconstructed.
 - **Learnable Decoders:** Reconstruction relies on a learnable decoder component, forming part of the foundation model.

Key Insight

Machine-learnable transforms create a latent representation optimized for robustness, invariance, and task-specific performance. These domains go beyond classical transforms, enabling applications in watermarking, hashing, and more.

Understanding Encoder Components in Deep Neural Networks

Key Elements of the Encoder Function

Encoder Definition

$$f_{\phi}(\mathbf{x}) = \sigma_K (\mathbf{W}_K \dots (\sigma_1 (\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_K)),$$

where $\phi = \{\mathbf{W}_1, \dots, \mathbf{W}_K; \mathbf{b}_1, \dots, \mathbf{b}_K\}$.

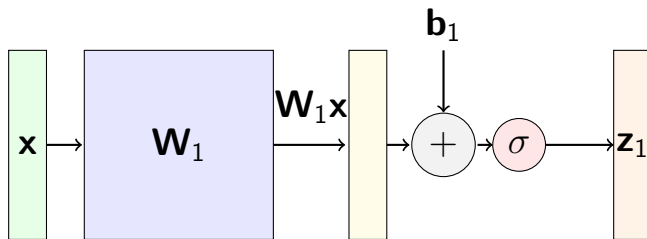
• Components:

- $\mathbf{W}_i$: Weight matrices for linear transformations.
- σ_i : Nonlinear activation functions (e.g., ReLU, sigmoid).
- $\mathbf{b}_i$: Bias terms for shifting activations.

• Why Nonlinearity Matters:

- Nonlinearities (σ_i) allow learning of complex features.
- Linear models (e.g., PCA) cannot achieve this.

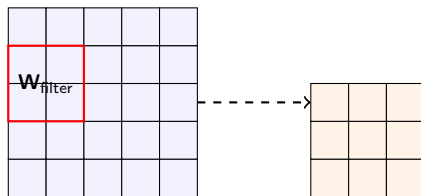
Matrix-Vector Multiplication and Non-Linearity (with Bias)



Explanation:

- The matrix $\mathbf{W}_1$ multiplies the input vector $\mathbf{x}$ to produce the intermediate vector $\mathbf{W}_1\mathbf{x}$.
- A bias $\mathbf{b}_1$ is added element-wise to the result.
- A point-wise non-linearity $\sigma(\cdot)$ is applied to yield the output vector $\mathbf{z}_1$.

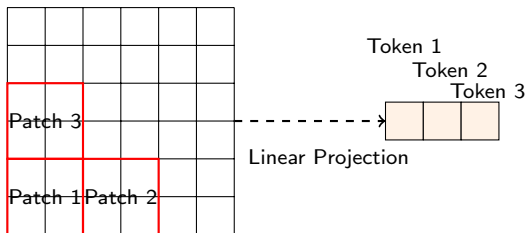
Convolutional Neural Networks (CNNs) - Sliding Window



Explanation:

- The sliding window (filter) moves across the input image, applying element-wise multiplication and summation.
- The resulting values are stored in the output feature map.

Image Patching for Transformers



Explanation:

- The image is divided into non-overlapping patches.
- Each patch is linearly projected into a token, forming the input sequence for the transformer model.

Self-Supervised Learning (SSL)

Learning Representations from One Modality

- **Objective:**

- Train the encoder using a single data modality (e.g., images).
- Use augmentations of the input $\mathbf{x}_0$ to create pairs for learning.

- **Process:**

- Apply transformations (e.g., cropping, rotation) to $\mathbf{x}_0$ to generate augmented views.
- Map augmented views to latent representations $\mathbf{y}_1$ and $\mathbf{y}_2$:

$$\mathbf{y}_1 = f_\phi(\text{augment}(\mathbf{x}_0)), \quad \mathbf{y}_2 = f_\phi(\text{augment}(\mathbf{x}_0)).$$

- Learn by maximizing similarity between $\mathbf{y}_1$ and $\mathbf{y}_2$.

- **Key Features:**

- Requires no labeled data.
- Relies on augmentations to create pseudo-labels for training.
- Produces a latent space $\mathbf{y}$ that is robust and representative.

Weakly Supervised Learning (WSL)

Learning Representations from Two Modalities

- **Objective:**

- Train the encoder using weakly aligned data from two modalities (e.g., images and text).
- Example: Image-text pairs $(\mathbf{x}_0, \mathbf{z})$ from a joint distribution $p(\mathbf{x}_0, \mathbf{z})$.

- **Process:**

- Use an encoder to map both modalities to their latent representations:

$$\mathbf{y}_x = f_\phi(\mathbf{x}_0), \quad \mathbf{y}_z = f_\phi(\mathbf{z}).$$

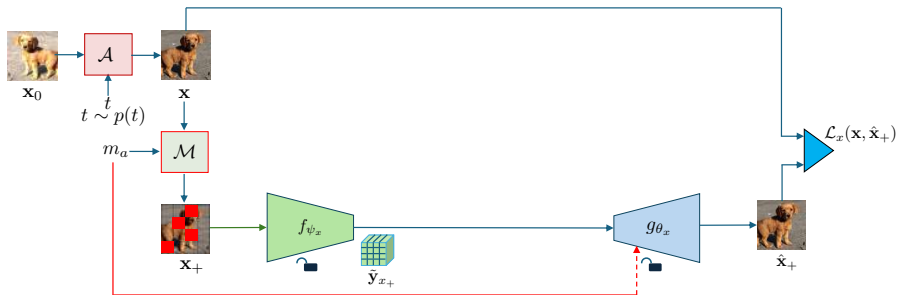
- Align $\mathbf{y}_x$ and $\mathbf{y}_z$ in the latent space, maximizing similarity for corresponding pairs.

- **Key Features:**

- Uses weak alignment (e.g., image-caption pairs) instead of strict labels.
- Produces a latent space $\mathbf{y}$ that bridges multiple modalities.
- Enables tasks like image-captioning or cross-modal retrieval.

Training of Foundation Models

Modern SSL FM architectures



Embedding-reconstruction (AE):

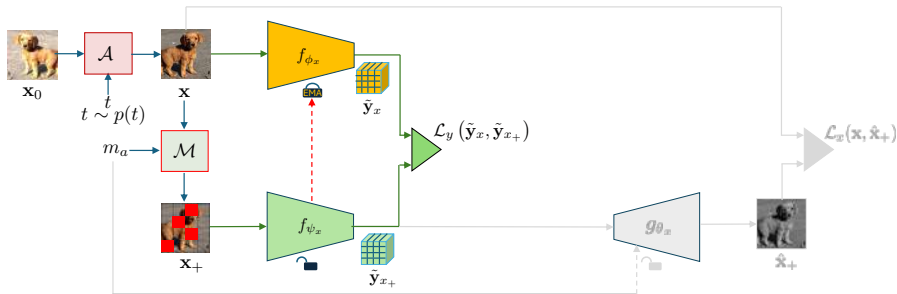
1. No mode collapse
2. High complexity
3. No good loss for pixel space

Denoising-AE, MAE

Training variations: scope (patches, aggregated patches, CLS); projectors; quantization

Training of Foundation Models

Modern SSL FM architectures



Embedding-reconstruction (AE): **Joint embedding:**

- | | |
|---------------------------------|--|
| 1. No mode collapse | 1. Possible mode collapse |
| 2. High complexity | 2. Low complexity |
| 3. No good loss for pixel space | 3. Some potentially good losses for latent space |

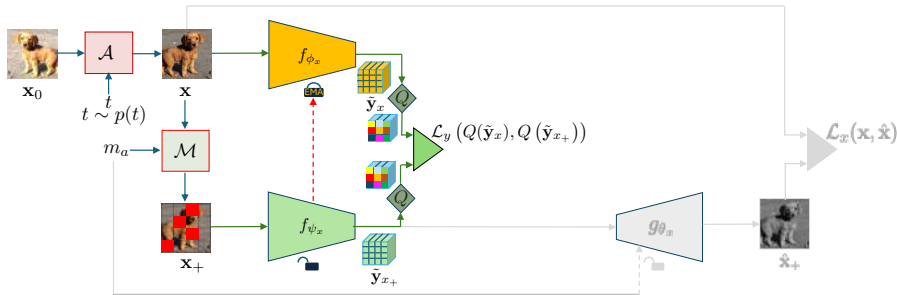
Denoising-AE, MAE

SimCLR, BYOL, Swav, MSN, DINO
VicReg, BarlowTwins

Training variations: scope (patches, aggregated patches, CLS); projectors; quantization

Training of Foundation Models

Modern SSL FM architectures



Embedding-reconstruction (AE): **Joint embedding:**

- | | |
|---------------------------------|--|
| 1. No mode collapse | 1. Possible mode collapse |
| 2. High complexity | 2. Low complexity |
| 3. No good loss for pixel space | 3. Some potentially good losses for latent space |

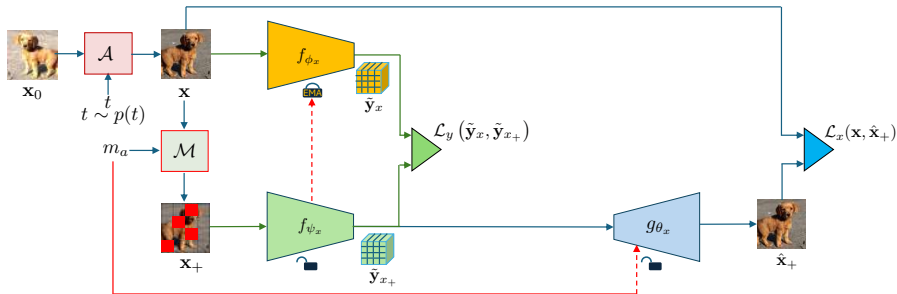
Denoising-AE, MAE

SimCLR, BYOL, Swav, MSN, DINO
VicReg, BarlowTwins

Training variations: scope (patches, aggregated patches, CLS); projectors; quantization

Training of Foundation Models

Modern SSL FM architectures



Hybrid versions: CAN, CAE, CMAE, BeIT

Embedding-reconstruction (AE): Joint embedding:

- | | |
|---------------------------------|--|
| 1. No mode collapse | 1. Possible mode collapse |
| 2. High complexity | 2. Low complexity |
| 3. No good loss for pixel space | 3. Some potentially good losses for latent space |

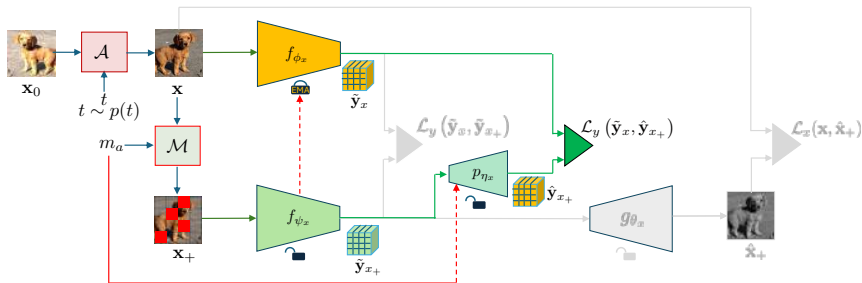
Denoising-AE, MAE

SimCLR, BYOL, Swav, MSN, DINO
VicReg, BarlowTwins

Training variations: scope (patches, aggregated patches, CLS); projectors; quantization

Training of Foundation Models

Modern SSL FM architectures



Embedding-reconstruction (AE): Joint embedding:

- | | |
|---------------------------------|--|
| 1. No mode collapse | 1. Possible mode collapse |
| 2. High complexity | 2. Low complexity |
| 3. No good loss for pixel space | 3. Some potentially good losses for latent space |

Denoising-AE, MAE

SimCLR, BYOL, Swav, MSN, DINO, I-JEPA, World Model
VicReg, BarlowTwins

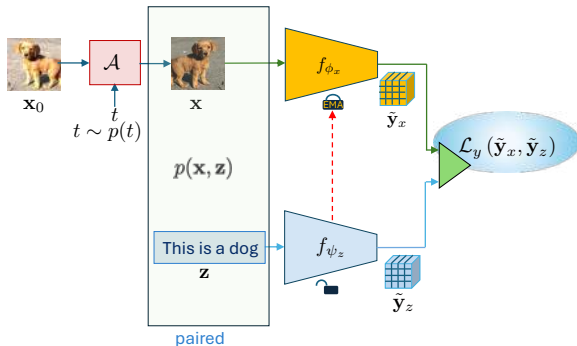
Joint embedding-prediction:

1. Possible mode collapse
2. Relatively low complexity
3. Some potentially good losses for latent space

Training variations: scope (patches, aggregated patches, CLS); projectors; quantization

Training of Foundation Models

Modern WSL FM architectures



Joint embedding:

1. Possible mode collapse
2. Low complexity
3. Some potentially good losses for latent space

CLIP, GLIP, BLIP, SigLIP, CoCa, ALIGN, etc.....

Part 3: Notations for digital watermarking and perceptual hashing

- **Embedding Function:**

$$\mathbf{y} = \phi(\mathbf{x}, \mathbf{k}) \quad (\text{Zero-bit watermarking})$$

$$\mathbf{y} = \phi(\mathbf{x}, \mathbf{k}, \mathbf{m}) \quad (\text{Multi-bit watermarking})$$

- Stego image $\mathbf{y}$ contains the embedded message $\mathbf{m}$.

Summary and Expectations

- Understand the notations ($\mathbf{x}$, $\mathbf{y}$, $\mathbf{w}$, $\mathbf{k}$, $\mathbf{m}$, etc.) and their significance.
- Familiarize yourself with image metrics (MSE, PSNR, WMSE, WPSNR).
- Prepare Python environment for homework and projects.
- Prepare for:
 - Theoretical foundations of multimedia security and privacy.
 - Hands-on coding projects to reinforce concepts.
 - Accessing course repositories with resources.
- **Next Lecture:** Begin exploring digital watermarking techniques.

Part 4: Foundations of signal detection in noise

Part 4.1: Key ideas

Hypotheses and Score Definition: main results

Hypothesis Testing:

- $H_0 : \mathbf{y} = \mathbf{x}$ (no watermark present).
- $H_1 : \mathbf{y} = \mathbf{x} + \mathbf{w}$ (watermark present).

Score Definition:

$$S = \frac{\mathbf{y}^T \mathbf{w}}{M}, \quad \text{where } M \text{ is the number of pixels in the image.}$$

By the Central Limit Theorem:

$$S \sim \mathcal{N}(\mu_{H_0}, \sigma_{H_0}^2) \quad \text{under } H_0,$$

$$S \sim \mathcal{N}(\mu_{H_1}, \sigma_{H_1}^2) \quad \text{under } H_1.$$

Probabilities and Gaussian Distributions

- **Probabilities:**

- **Probability of Correct Detection (P_D):** $P_D(\gamma) = \Pr(S \geq \gamma \mid H_1)$
- **Probability of False Acceptance (P_{FA}):** $P_{FA}(\gamma) = \Pr(S \geq \gamma \mid H_0)$

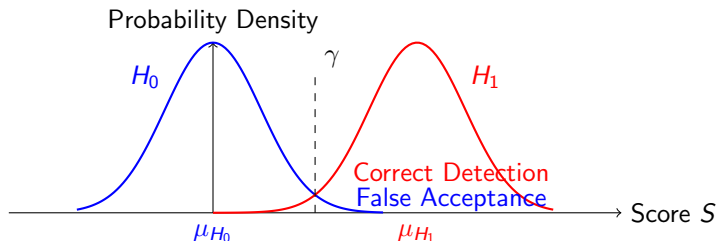
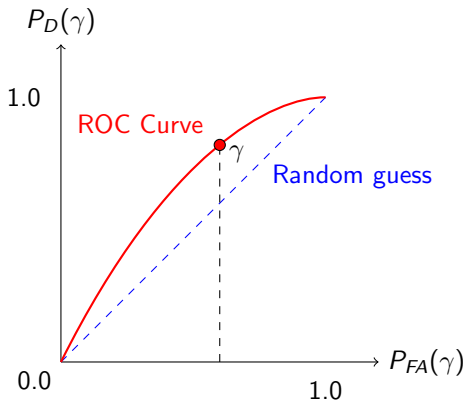


Figure: Gaussian Distributions under H_0 and H_1 with Threshold γ

Receiver Operating Characteristic (ROC) Curve

- The ROC curve evaluates the trade-off between (as a function of γ):
 - **Probability of Correct Detection (P_D)**: Maximize.
 - **Probability of False Acceptance (P_{FA})**: Minimize.
- **Ideal ROC**: Achieve $P_D \rightarrow 1$ and $P_{FA} \rightarrow 0$.
- The diagonal represents random guessing (worst-case scenario).



**Part 4.2: Derivation of score and its statistics under
different hypothesis**

Hypothesis Testing: derivation of score

Hypotheses:

- $H_0 : \mathbf{y} = \mathbf{x}$ (no watermark present).
- $H_1 : \mathbf{y} = \mathbf{x} + \mathbf{w}$ (watermark present).

Assumptions:

- $\mathbf{x}$ is a **zero-mean Gaussian vector** with covariance $\sigma_x^2 \mathbf{I}_M$.
- $\mathbf{w}$ is a **known watermark vector** with $\|\mathbf{w}\|_2^2 = E_w$.
- $\mathbf{y}$ is the observed vector.

Goal: Derive the test statistic $S(\mathbf{y})$ and compute:

- Probability of false acceptance ($P_{FA}(\gamma)$).
- Probability of correct detection ($P_D(\gamma)$).
- Probability of miss ($P_{\text{miss}}(\gamma) = 1 - P_D(\gamma)$).

Likelihood Ratio and Score Definition

Definition

Likelihood Ratio:

$$\Lambda(\mathbf{y}) = \frac{p(\mathbf{y}|H_1)}{p(\mathbf{y}|H_0)} \underset{H_0}{\overset{H_1}{\gtrless}} \gamma.$$

Densities:

$$p(\mathbf{y}|H_0) = \frac{1}{(2\pi\sigma_x^2)^{M/2}} \exp\left(-\frac{\mathbf{y}^\top \mathbf{y}}{2\sigma_x^2}\right),$$

$$p(\mathbf{y}|H_1) = \frac{1}{(2\pi\sigma_x^2)^{M/2}} \exp\left(-\frac{(\mathbf{y} - \mathbf{w})^\top (\mathbf{y} - \mathbf{w})}{2\sigma_x^2}\right).$$

Log-Likelihood Ratio:

$$\ln \Lambda(\mathbf{y}) = \frac{2\mathbf{y}^\top \mathbf{w} - \mathbf{w}^\top \mathbf{w}}{2\sigma_x^2}.$$

Score:

$$S(\mathbf{y}) = \frac{1}{M} \mathbf{y}^\top \mathbf{w}.$$

Derivation of Mean Under H_0

Assumptions:

- $H_0 : \mathbf{y} = \mathbf{x}$ (no watermark present).
- $\mathbf{x}$ is a zero-mean Gaussian vector with covariance $\sigma_x^2 \mathbf{I}_M$.
- The score is defined as:

$$S(\mathbf{y}) = \frac{1}{M} \mathbf{y}^\top \mathbf{w}.$$

Mean Under H_0 :

$$\mathbb{E}[S(\mathbf{y})|H_0] = \mathbb{E} \left[\frac{1}{M} \mathbf{x}^\top \mathbf{w} \middle| H_0 \right].$$

Linearity of Expectation

Since $\mathbf{x}$ is zero-mean ($\mathbb{E}[\mathbf{x}] = 0$):

$$\mathbb{E}[S(\mathbf{y})|H_0] = \frac{1}{M} \mathbb{E}[\mathbf{x}^\top \mathbf{w}] = \frac{1}{M} \mathbf{w}^\top \mathbb{E}[\mathbf{x}] = 0.$$

Result:

$$\mu_0 = 0.$$

Derivation of Variance Under H_0

Variance Under H_0 :

$$\text{Var}(S(\mathbf{y})|H_0) = \mathbb{E}[S(\mathbf{y})^2|H_0] - \mathbb{E}[S(\mathbf{y})|H_0]^2.$$

$$\text{Var}(S(\mathbf{y})|H_0) = \mathbb{E} \left[\left(\frac{1}{M} \mathbf{x}^\top \mathbf{w} \right)^2 \middle| H_0 \right].$$

Step 1: Expand the Quadratic Form

$$\mathbb{E} \left[\left(\frac{1}{M} \mathbf{x}^\top \mathbf{w} \right)^2 \middle| H_0 \right] = \frac{1}{M^2} \mathbb{E} [(\mathbf{w}^\top \mathbf{x})^2 | H_0].$$

Step 2: Variance of Linear Combination of Gaussians

$$\mathbf{w}^\top \mathbf{x} \sim \mathcal{N}(0, \|\mathbf{w}\|_2^2 \sigma_x^2) \text{ thus } \mathbb{E} [(\mathbf{w}^\top \mathbf{x})^2 | H_0] = \|\mathbf{w}\|_2^2 \sigma_x^2.$$

Step 3: Compute Variance

$$\text{Var}(S(\mathbf{y})|H_0) = \frac{\|\mathbf{w}\|_2^2 \sigma_x^2}{M^2}.$$

Result:

$$\sigma_0^2 = \frac{\|\mathbf{w}\|_2^2 \sigma_x^2}{M^2}.$$

Derivation of Mean Under H_1

Assumptions:

- $H_1 : \mathbf{y} = \mathbf{x} + \mathbf{w}$ (watermark present).
- $\mathbf{x}$ is a zero-mean Gaussian vector with covariance $\sigma_x^2 \mathbf{I}_M$.
- The score is defined as:

$$S(\mathbf{y}) = \frac{1}{M} \mathbf{y}^\top \mathbf{w}.$$

Mean Under H_1 :

$$\mathbb{E}[S(\mathbf{y})|H_1] = \mathbb{E} \left[\frac{1}{M} (\mathbf{x} + \mathbf{w})^\top \mathbf{w} \middle| H_1 \right].$$

Separate Expectation Terms

$$\mathbb{E}[S(\mathbf{y})|H_1] = \frac{1}{M} (\mathbb{E}[\mathbf{x}^\top \mathbf{w}] + \mathbb{E}[\mathbf{w}^\top \mathbf{w}]).$$

$$\mathbb{E}[S(\mathbf{y})|H_1] = \frac{1}{M} (0 + \|\mathbf{w}\|_2^2) = \frac{\|\mathbf{w}\|_2^2}{M}.$$

Result:

$$\mu_1 = \frac{\|\mathbf{w}\|_2^2}{M}.$$

Derivation of Variance Under H_1

Variance Under H_1 :

$$\text{Var}(S(\mathbf{y})|H_1) = \mathbb{E}[S(\mathbf{y})^2|H_1] - \mathbb{E}[S(\mathbf{y})|H_1]^2.$$

Step 1: Expand the Quadratic Form

$$\mathbb{E}[S(\mathbf{y})^2|H_1] = \frac{1}{M^2} \mathbb{E}[(\mathbf{w}^\top \mathbf{x})^2 + 2(\mathbf{w}^\top \mathbf{x})(\mathbf{w}^\top \mathbf{w}) + (\mathbf{w}^\top \mathbf{w})^2 | H_1].$$

Step 2: Simplify Terms

- $\mathbb{E}[(\mathbf{w}^\top \mathbf{x})^2 | H_1] = \|\mathbf{w}\|_2^2 \sigma_x^2.$
- $\mathbb{E}[(\mathbf{w}^\top \mathbf{x})(\mathbf{w}^\top \mathbf{w}) | H_1] = 0$ (independence).
- $\mathbb{E}[(\mathbf{w}^\top \mathbf{w})^2 | H_1] = (\mathbf{w}^\top \mathbf{w})^2 = \|\mathbf{w}\|_2^4.$

Step 3: Compute Variance

$$\mathbb{E}[S(\mathbf{y})^2 | H_1] = \frac{1}{M^2} (\|\mathbf{w}\|_2^2 \sigma_x^2 + \|\mathbf{w}\|_2^4).$$

$$\text{Var}(S(\mathbf{y})|H_1) = \frac{\|\mathbf{w}\|_2^2 \sigma_x^2}{M^2}.$$

Result:

$$\sigma_1^2 = \frac{\|\mathbf{w}\|_2^2 \sigma_x^2}{M^2}.$$

Distribution of the Score $S(\mathbf{y})$: summary

Under H_0 :

$$S(\mathbf{y}) = \frac{1}{M} \mathbf{x}^\top \mathbf{w}.$$

- Mean: $\mu_0 = 0$.
- Variance: $\sigma_0^2 = \frac{\|\mathbf{w}\|_2^2 \sigma_x^2}{M^2}$.

Under H_1 :

$$S(\mathbf{y}) = \frac{1}{M} (\mathbf{x} + \mathbf{w})^\top \mathbf{w}.$$

- Mean: $\mu_1 = \frac{\|\mathbf{w}\|_2^2}{M}$.
- Variance: $\sigma_1^2 = \frac{\|\mathbf{w}\|_2^2 \sigma_x^2}{M^2}$.

Key Observation

Both $S(\mathbf{y})$ distributions are Gaussian, differing only in their means.

Definition of Q-Function

Definition of $Q(x)$

$$Q(x) = \frac{1}{\sqrt{2\pi}} \int_x^{\infty} e^{-\frac{t^2}{2}} dt$$

The Q -function represents the **right tail probability** of a standard normal distribution.

Python Computation:

Python Code

```
from scipy.stats import norm
Q_value = 1 - norm.cdf(x) # Computes Q(x)
```

General Formulas for Q-Function

1. Probability of False Acceptance (P_{FA}):

$$P_{FA}(\gamma) = Q\left(\frac{\gamma - \mu_0}{\sigma_0}\right)$$

- μ_0 : Mean of $S(\mathbf{y})$ under H_0 .
- σ_0^2 : Variance of $S(\mathbf{y})$ under H_0 .

2. Probability of Miss (P_{miss}):

$$P_{miss}(\gamma) = 1 - Q\left(\frac{\gamma - \mu_1}{\sigma_1}\right)$$

- μ_1 : Mean of $S(\mathbf{y})$ under H_1 .
- σ_1^2 : Variance of $S(\mathbf{y})$ under H_1 .

Important Notes

- The right tail ($Q(x)$) corresponds to P_{FA} .
- The complement of the left tail ($1 - Q(x)$) corresponds to P_{miss} .

Error Probabilities

Probability of False Acceptance ($P_{FA}(\gamma)$):

$$P_{FA}(\gamma) = P(S(\mathbf{y}) > \gamma | H_0) = Q\left(\frac{\gamma - \mu_0}{\sigma_0}\right),$$

$$P_{FA}(\gamma) = Q\left(\frac{\gamma}{\sqrt{\frac{\|\mathbf{w}\|_2^2 \sigma_x^2}{M^2}}}\right).$$

Probability of Correct Detection ($P_D(\gamma)$):

$$P_D(\gamma) = P(S(\mathbf{y}) > \gamma | H_1) = Q\left(\frac{\gamma - \mu_1}{\sigma_1}\right),$$

$$P_D(\gamma) = Q\left(\frac{\gamma - \frac{\|\mathbf{w}\|_2^2}{M}}{\sqrt{\frac{\|\mathbf{w}\|_2^2 \sigma_x^2}{M^2}}}\right).$$

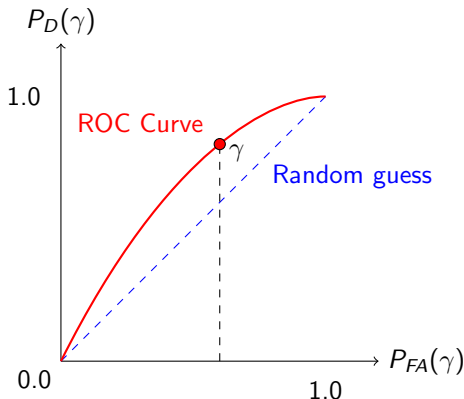
Probability of Miss ($P_{miss}(\gamma)$):

$$P_{miss}(\gamma) = 1 - P_D(\gamma).$$

ROC Curve

Definition of ROC curve

The Receiver Operating Characteristic (ROC) curve plots $P_D(\gamma)$ versus $P_{FA}(\gamma)$ as γ varies.



Summary of Results

Key Results:

- Score Definition: $S(\mathbf{y}) = \frac{1}{M} \mathbf{y}^\top \mathbf{w}$.
- Distributions:
 - Under H_0 : $\mu_0 = 0$, $\sigma_0^2 = \frac{\|\mathbf{w}\|_2^2 \sigma_x^2}{M^2}$.
 - Under H_1 : $\mu_1 = \frac{\|\mathbf{w}\|_2^2}{M}$, $\sigma_1^2 = \frac{\|\mathbf{w}\|_2^2 \sigma_x^2}{M^2}$.

Important Notes

- Error Probabilities:

$$P_{\text{FA}}(\gamma) = Q \left(\frac{\gamma}{\sqrt{\frac{\|\mathbf{w}\|_2^2 \sigma_x^2}{M^2}}} \right)$$

$$P_D(\gamma) = Q \left(\frac{\gamma - \frac{\|\mathbf{w}\|_2^2}{M}}{\sqrt{\frac{\|\mathbf{w}\|_2^2 \sigma_x^2}{M^2}}} \right),$$

$$P_{\text{miss}}(\gamma) = 1 - P_D(\gamma).$$

Part 4.5: Threshold γ selection under two strategies

- **Neyman-Pearson Strategy:**

- Optimize the threshold γ under a fixed constraint on the **probability of false acceptance** (P_{FA}).
- Minimize the **probability of miss** ($P_{\text{miss}} = 1 - P_D$) as a function of γ .

- **Bayesian Strategy:**

- Minimize the **average probability of error**, defined as:

$$P_{\text{error}} = P_{FA}P(H_0) + P_{\text{miss}}P(H_1),$$

where $P(H_0)$ and $P(H_1)$ are the prior probabilities of the null and alternative hypotheses.

- In symmetric cases ($P(H_0) = P(H_1) = 0.5$), simplify to:

$$P_{\text{error}} = \frac{1}{2}(P_{FA} + P_{\text{miss}}).$$

Neyman-Pearson Strategy

Goal

Given a constraint on the **probability of false acceptance** (P_{FA}), find the optimal γ to minimize the **probability of miss** (P_{miss}).

Decision Rule:

$$\Lambda(\mathbf{x}) = \frac{p(\mathbf{x}|H_1)}{p(\mathbf{x}|H_0)} \underset{H_0}{\overset{H_1}{\gtrless}} \gamma,$$

where:

- $p(\mathbf{x}|H_1)$: Likelihood of $\mathbf{x}$ under H_1 .
- $p(\mathbf{x}|H_0)$: Likelihood of $\mathbf{x}$ under H_0 .
- γ : Decision threshold.

Optimization:

- **Fix** P_{FA} :

$$P_{FA}(\gamma) = \int_{\Lambda(\mathbf{x}) > \gamma} p(\mathbf{x}|H_0) d\mathbf{x}.$$

- **Minimize** P_{miss} :

$$P_{miss}(\gamma) = \int_{\Lambda(\mathbf{x}) \leq \gamma} p(\mathbf{x}|H_1) d\mathbf{x}.$$

Goal

Minimize the **average probability of error** :

$$P_{\text{error}}(\gamma) = P_{FA}(\gamma)P(H_0) + P_{\text{miss}}(\gamma)P(H_1).$$

Key Assumptions:

- Prior probabilities $P(H_0)$ and $P(H_1)$.
- Symmetric case ($P(H_0) = P(H_1) = 0.5$) simplifies to:

$$P_{\text{error}}(\gamma) = \frac{1}{2}(P_{FA}(\gamma) + P_{\text{miss}}(\gamma)).$$

Optimization:

- For Gaussian distributions ($\mathcal{N}(\mu_0, \sigma^2)$ under H_0 , $\mathcal{N}(\mu_1, \sigma^2)$ under H_1):

$$\gamma_{\text{opt}} = \frac{\mu_0 + \mu_1}{2}.$$

Comparison of Probability and Gaussian Distributions

Distributions and Parameters: x is considered as the score S_y

$$p(x|H_0) = \mathcal{N}(\mu_0, \sigma_0^2), \quad \mu_0 = 0, \sigma_0 = 1, P(H_0) = 0.5,$$

$$p(x|H_1) = \mathcal{N}(\mu_1, \sigma_1^2), \quad \mu_1 = 2, \sigma_1 = 1, P(H_1) = 1 - P(H_0).$$

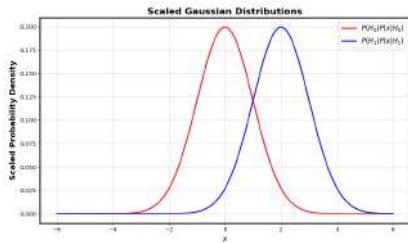


Figure: Probabilities vs Threshold (γ).

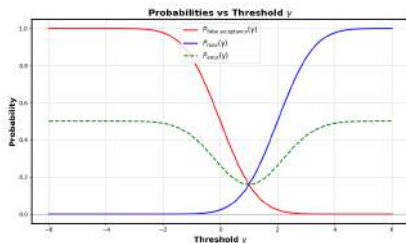


Figure: Scaled Gaussian Distributions.

Comparison of Probability and Gaussian Distributions

Distributions and Parameters:

$$p(x|H_0) = \mathcal{N}(\mu_0, \sigma_0^2), \quad \mu_0 = 0, \sigma_0 = 1, P(H_0) = 0.8,$$

$$p(x|H_1) = \mathcal{N}(\mu_1, \sigma_1^2), \quad \mu_1 = 2, \sigma_1 = 1, P(H_1) = 1 - P(H_0).$$

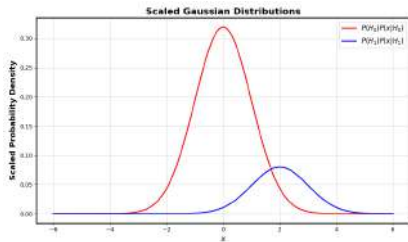


Figure: Probabilities vs Threshold (γ).

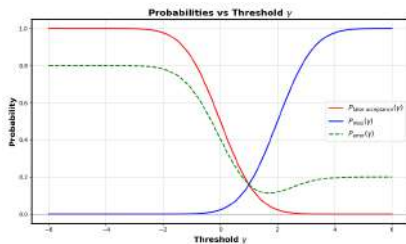


Figure: Scaled Gaussian Distributions.

Comparison of Probability and Gaussian Distributions

Distributions and Parameters:

$$p(x|H_0) = \mathcal{N}(\mu_0, \sigma_0^2), \quad \mu_0 = 0, \sigma_0 = 1, P(H_0) = 0.5,$$
$$p(x|H_1) = \mathcal{N}(\mu_1, \sigma_1^2), \quad \mu_1 = 2, \sigma_1 = 5, P(H_1) = 1 - P(H_0).$$

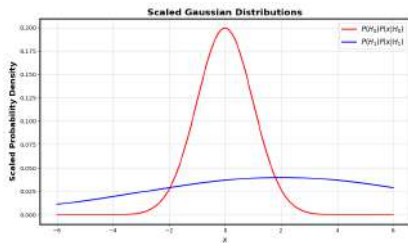


Figure: Probabilities vs Threshold (γ).

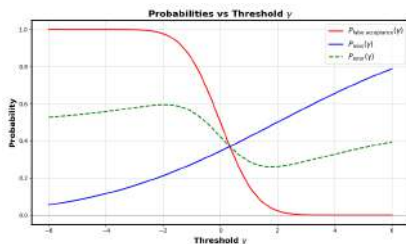


Figure: Scaled Gaussian Distributions.